

Auditoría A05: Arquitectura del *pipeline* y run.sh

Informe del Entregable 1, Sección 5 (informe_e1_smnv3.tex, commit 330fd2e)

Revisión: 28 de septiembre de 2026

1. Objeto

Se audita la Sección 5 (“*Pipeline*”: principios, “Script principal `run.sh`” y flujograma). Por instrucción del autor, la información de *uso* del proyecto (instalación, argumentos, ejemplos, tiempos, salidas) sale del informe y pasa a un documento de anexos separado, escrito para un usuario básico de Linux. Esta auditoría define qué queda en el informe, qué pasa al anexo y qué debe corregirse. Todo se contrastó con `run.sh`, `environment.yml` y los scripts auxiliares en el commit 8ee6184.

Actualización del 29-09-2026 (decisiones del autor). El flujograma pasó a la Sección 6 (“Obtención y procesamiento de los datos”), que ahora es un párrafo con las acciones realizadas y el flujograma explicado brevemente. La Sección 5 conserva los principios de diseño, sin remitir a pasos individuales.

2. Comportamiento verificado de run.sh (commit 8ee6184)

1. Activa por sí mismo el entorno conda `smn_toe` (o `e1_smn`, el nombre anterior) si existe, y antepone su `bin` al `PATH`, por los clústeres HPC donde el `python3` del sistema tiene prioridad.
2. Valida los argumentos `[STEP_FROM]` `[STEP_TO]` `[MAX_MODELS]`; los pasos válidos son 00 00b 01 02 03 04 05 06 07.
3. Muestra un **reporte *preflight*** (`preflight_report.py`): qué artefactos existen, cuáles faltan y cuánto tardaron en corridas anteriores en esa máquina. El informe original no lo menciona.
4. Ejecuta los pasos del rango pedido, registra la salida en `logs/pipeline.log` y la duración de cada paso en `logs/step_timings.csv`.
5. Siempre, sin importar el rango: registro de modelos, cinco scripts de gráficos, reporte de estado (`logs/run_report_<fecha>.txt`) y paquete `informe/entregable_update.tar.gz`.

Hay dos preguntas interactivas, ambas con límite de 15s. La de buscar en nodos alternativos y Copernicus la hace `02_download_all_sources.sh`, no `run.sh`; solo aparece con terminal interactiva y modelos pendientes, y el silencio equivale a “sí”. La de regenerar las figuras existentes la hace `plot_common.py`; el silencio equivale a “no”.

Recursos medidos en esta máquina (`logs/step_timings.csv`, `logs/download_timings.csv`): Paso 00b, mediana de 95 min; Paso 01, hasta 45 min; Paso 02 (descarga de 160 combinaciones), hasta ~13 h en una corrida, con 164 descargas medidas, mediana de 3,5 min y máximo de 84 min cada una; Paso 04, hasta 49 min; Pasos 05–07, pocos minutos. Espacio: `data/raw/cmip6` 73 GB, `data/interim` 4,7 GB y `data/processed/masked` 4,3 GB (esta última medida con la ventana ampliada del E2).

3. Hallazgos

ID	Sev.	Ubicación	Hallazgo y evidencia	Corrección aplicada
A05-1	Media	Títulos 5 y 6	Dos secciones consecutivas con el mismo título, “ <i>Pipeline</i> ”.	“Arquitectura del <i>pipeline</i> ” (5) y “Descripción de los pasos” (6).

ID	Sev.	Ubicación	Hallazgo y evidencia	Corrección aplicada
A05-2	Media	5.1	El detalle de uso de <code>run.sh</code> (argumentos, ejemplos, guardia de descargas, reportes) ocupa unas 600 palabras en el cuerpo del informe.	Por instrucción del autor pasa al Anexo; el informe conserva un párrafo sobre el papel de <code>run.sh</code> y remite al anexo.
A05-3	Media	5 (principios)	“un <code>git clone</code> nuevo más <code>./run.sh</code> , sin intervención manual, obtiene todo lo que sea posible”. Omite dos requisitos previos (crear el entorno conda; credenciales de Copernicus para 02c) y que desde <code>main</code> se obtiene el estado del E2, no el del E1 (ver A01-6).	En el informe la frase se reemplaza por los tres principios; los requisitos y la versión que se debe usar se explican en el Anexo.
A05-4	Media	5 (principios)	La idempotencia se presenta como absoluta, pero el Paso 06 la abandona a propósito desde <code>c07f4c5</code> .	Se nombra la excepción y su razón.
A05-5	Media	5.1	No menciona el reporte <i>preflight</i> , la activación automática del entorno ni el registro de tiempos, que son lo primero que ve un usuario al ejecutar <code>run.sh</code> .	Se documentan en el Anexo.
A05-6	Baja	Flujograma	El código TikZ conserva nodos comentados (“Notebook”, “Reporte + paquete final”) y un comentario erróneo (<code>scale=0.95% reduce todo al 75%</code>). El pie menciona el <i>notebook</i> de gráficos exploratorios, que dejó de versionarse en <code>9ba3f2c</code> y que el diagrama no dibuja.	Código limpio; la última caja pasa a ser “Gráficos, reporte y paquete”; el pie se ajusta a lo dibujado.
A05-7	Baja	Código (fuera del informe)	El comentario de cabecera de <code>run.sh</code> está desactualizado: nombra LLNL como nodo principal de 00b (ahora es ORNL), <code>model_availability_report.*</code> como salida de 00b (ahora es <code>model_availability_priority.csv</code>) y el <i>notebook</i> como vía de gráficos.	No se modifica código en esta revisión. Se recomienda actualizarlo junto con el cierre del E1 (ver A10).
A05-8	–	5.1	Verificado sin cambios: orden fijo de pasos; <code>logs/pipeline.log</code> ; invocación del Paso 02 a través de <code>run_download_if_idle.sh</code> con <code>pgrep</code> ; cinco scripts <code>plot_*.py</code> ; reporte de estado y paquete siempre al final.	Se conservan (en el informe o en el Anexo).

4. Reparto de contenido entre informe y anexo

Queda en el informe (Sección 5)	Pasa al Anexo de uso
Repositorio, punto de entrada, salidas en <code>data/processed/</code> ; tres principios de diseño; papel de <code>run.sh</code> en un párrafo; flujograma.	Requisitos (Linux, conda, disco, red, credenciales CDS); instalación; argumentos y ejemplos; qué muestra el <i>preflight</i> ; preguntas interactivas; tiempos; carpetas de salida; cómo leer el reporte de estado; problemas frecuentes; cómo reproducir el estado del E1.

5. Extensión

Original: 749 palabras (incluido el flujograma). Propuesto para el informe: 307 palabras. El resto se traslada al Anexo, donde se amplía con los detalles que necesita un usuario básico.

6. Texto propuesto

Inicio del texto propuesto para el informe revisado

6.1. Arquitectura del *pipeline*

El *pipeline* vive en el repositorio `smn_toe`, tiene un único punto de entrada (`run.sh`) y deja sus salidas en `data/processed/`, de donde el Entregable 2 las lee sin necesidad de conocer el detalle interno de este módulo. Se rige por tres principios:

- **Idempotencia.** Cada paso omite el procesamiento cuya salida ya existe, de modo que una corrida puede interrumpirse y reanudarse sin recalcular lo hecho. La excepción deliberada es el control de calidad, que se recalcula completo en cada corrida porque es barato y un resultado en caché puede quedar obsoleto.
- **Verificación por valor y no por metadato.** Las unidades, los valores de relleno y la lista de meses se comprueban sobre los datos numéricos y no sobre los atributos declarados en los archivos. Existen archivos cuyo atributo contradice su contenido; por ejemplo, archivos que declaran grados Celsius y contienen valores en Kelvin.
- **Cascada de fuentes con degradación explícita.** Si el nodo principal de ESGF no resuelve un modelo, se prueban automáticamente nodos alternativos y Copernicus CDS. Lo que ninguna fuente resuelve queda registrado en un reporte de estado, en lugar de fallar en silencio.

`run.sh` no contiene lógica de procesamiento propia: ejecuta los pasos 00, 00b, 01, 02, 03, 04, 05, 06 y 07 en ese orden fijo, o solo un rango de ellos, y registra la salida de cada uno en `logs/pipeline.log`. Al terminar cualquier corrida, completa o parcial, genera siempre el registro de modelos, las figuras automáticas, el reporte de estado y un paquete comprimido con los artefactos para actualizar el informe. Así, cada ejecución deja constancia verificable de qué modelos quedaron descargados, procesados o pendientes. La instalación, los argumentos de `run.sh`, los tiempos y el espacio en disco requeridos, y la interpretación de las salidas se describen en el documento de anexos que acompaña a este informe.

Fin del texto propuesto
